CMSC433 / Part 02

Functor, Applicative, and State types

Guided practice

Use with the chat: work through one question at a time.

Idea. `fmap` changes the value inside a context. For `State s a`, `s` is the stored type and `a` is the returned type.

Worked example. `null <$> get` has type `State [a] Bool`: `null :: [a] -> Bool` forces the stored state to be a list; the result is a Boolean and the State context stays in place.

1. A function inside Maybe

Infer `(,) <$> Just False`. First write the type of `(,)` after supplying a Boolean, then retain the outer context.

Answer: Intermediate function :: [ ]; whole expression :: [ ].

2. Apply a contextual function

Infer `(++) <$> Just "a" <*> Just "bc"`. Give the type immediately before `<*>` and the whole type.

Answer: [Intermediate type, final type, and why <*> fits.]

3. Track the state separately

Infer `get >>= \xs -> put (reverse xs)`.

Answer: Stored type: [ ]; returned type: [ ]; whole type: [ ].

4. Changed independent question

Infer `\f -> modify f >> get`. Explain the relationship between the input and output types of `f`.

Answer: [Most general type and reasoning.]

Source: Sample Q1, p. 1; Monads Functor/Applicative section; State interface.